

Encapsulation & access modifiers

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Understanding encapsulation

- Encapsulation can be explained with a mobile device
- You only have access to dedicated exposed features like screen, speaker, etc
- The inner workings are entirely hidden from users



Public properties

- Properties marked `public` can be accessed by object instances

```
class Car {  
    // Public property  
    public String color;  
  
    // Public constructor  
    public Car(String color){  
        this.color = color;  
    }  
}
```

```
public class Main {  
  
    // main method  
    public static void main(String[] args) {  
        Car myCar = new Car("brown");  
        // Can access public members  
        // from object instance  
        System.out.println(  
            myCar.color); // Brown  
    }  
}
```

Private properties

- Properties marked `private` can be accessed by object instances
- Properties marked `private` can only be used within the class

```
// Car class
class Car {
    public String color;
    // Private properties
    private String model;

    public Car(String color){
        this.color = color;
    }
}
```

```
// Main class
public class Main {
    // main method
    public static void main(String[] args) {
        Car myCar = new Car("brown");
        // Calling private properties causes errors
        System.out.println(
            myCar.model); // Java compilation error
    }
}
```

Public methods

```
// Car class
class Car {
    public String color;
    private String model;

    public Car(String color){
        this.color = color;
    }

    // Public method
    public String getModel(){
        return this.model;
    }
}
```

Private methods

```
class Car {  
  
    // Private method, "calculateSpeed" can only be used within "Car" class  
    private calculateSpeed(){  
        // Trademarked formula code  
    }  
  
    public int getSpeed(){  
        // "calculateSpeed" can be used anywhere within "Car" class  
        return this.calculateSpeed();  
    }  
}
```

Static methods

- Properties marked `static` can be accessed without an object instance
- Typically used to house shared code/libraries

```
// Formula class
static class Formula {

    // Method for calculating square
    static int getSquare(int number) {
        return number * number;
    }
}
```

```
// Main class
public class Main {

    public static void
        main(String[] args) {
        // We can use "getSquare"
        // without an object instance
        System.out.println(
            Formula.getSquare(5)); // 25
        }
}
```


Recap

- Access modifiers: `public` , `private` , `static`
- Access modifiers help secure applications
- `static` enables the use of classes without creating instances

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Centralising code with inheritance

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Working with multiple classes

```
// Toyota class
class Toyota {
    public String model;
    public String color;
    public String licensePlate;
}
```

```
// Honda class
class Honda {
    public String model;
    public String color;
    public String licensePlate;
}
```

```
// Mercedes class
class Mercedes {
    public String model;
    public String color;
    public String licensePlate;
}
```

- Duplication is common when working with classes

The power of inheritance

- Inheritance solved the problem of duplicate code
- All shared code can be in one place
- Classes can then inherit using the `extends` keyword

```
// Car class
class Car {
    // All public properties
    // are inherited
    public String model;
    public String color;
    public String licensePlate;
}
```

```
// Toyota inheriting from Car class
class Toyota extends Car {

}
```

```
// Honda inheriting from Car class
class Honda extends Car {

}
```

```
// Mercedes inheriting from Car class
class Mercedes extends Car {

}
```

Introducing base class

- The Base class is the class that contains shared code
- The `Car` class is an example of a base class

```
class Car {  
    public String model;  
    public String color;  
    public String licensePlate;  
}
```

Inheriting with subclasses

- The subclass is the class that inherits the base class
- The `Toyota` class is an example of a subclass
- A subclass inherits all public properties & methods of the base class
- The `super()` method calls the constructor of the base class

```
// Car Base Class
class Car {
    public String model;
    public String color;
    public String licensePlate;
}
```

```
// Toyota subclass
class Toyota extends Car {

    // Toyota constructor
    public Toyota(){
        // super() calls
        // baseclass constructor (Car)
        super();
    }
}
```

Creating inheritance instances

- Properties from the base class can be used by object instances

```
// Car class
class Car {
    public String color;

    // Car constructor
    public Car(String color){
        this.color = color;
    }
}
```

```
// Toyota subclass
class Toyota extends Car {
    public Toyota(){
        super();
    }
}
```

```
// Main class
public class Main {
    public static void main(String[] args) {
        Toyota myToyota = new Toyota("Brown");
        // "color" property inherited from "Car"
        System.out.println(
            myToyota.color); // Brown
    }
}
```


Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Making the car class abstract

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Sani Yusuf

Lead Software Engineering Content
Developer

Explaining abstract class

- The phrase Car only exists as a placeholder
- We still need to know more like what type of car it is



Food analogy for abstract class

- You never ask for just **food** at a restaurant
- You always specify what type of food you want to eat
 - bread
 - pasta
 - rice



Understanding abstract classes

- Abstract classes exist to hold features for inheriting classes
- Abstract classes cannot have object instances created
- Abstract classes generally do not have code implementation

Car abstract class

```
// Car abstract class
abstract class Car {
    // abstract drive method
    abstract void drive(); // No Code Implementation
}
```

Abstract methods

- Abstract method have no implementation
- The class that inherits, must implement abstract methods

```
// Car abstract class
abstract class Car {
    // abstract drive method
    abstract void drive(); // No Code
                                // implementation
}
```

```
// Toyota class
class Toyota extends Car {
    void drive(){
        // Toyota drive() implementation
    }
}
```

```
// Porsche class
class Porsche extends Car {
    void drive(){
        // Porsche drive() implementation
    }
}
```

Creating concrete methods

- Concrete methods are the methods with code implementation
- Subclasses do not need to implement concrete methods

```
// abstract Car class
abstract class Car {
    private int topSpeed;

    // Concrete method with implementation
    public getTopSpeed(){
        return this.topSpeed;
    }

    abstract void drive();
    // No Code Implementation
}
```


Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA